

About Me

I am Vivek Singh currently a third-year undergraduate pursuing Computer Science at the Indian Institute of Information Technology Nagpur with a CGPA of 8.08/10.0 (expected graduation: June 2026). My technical toolkit includes languages like Flutter, C++, Kotlin, Dart, JavaScript, and TypeScript, which I leverage to build responsive applications. As App Development Lead in GDSC i have collaborated with open source team building applications for the institution.

Project Title: LED Badge Magic App

Possible Mentors: Aditya, MarioB, cweitat.

Why This Project?

I picked this project because it lets me mix creativity with technology in a real, hands-on way. I'm excited to work with Flutter to make LED badges more fun and customizable, and I believe this project will let me learn a lot while making a useful tool for the community.

Abstract

Flutter Badge Magic is a mobile app that lets you design custom LED badges. With this app, you can create moving text, add clipart, and build simple animations. It connects to your LED badge via Bluetooth, turning your designs into a lively display.

Contribution:

1. I was able to successfully add the feature for fontstyle support [Github](#)
2. I have a draft pr for both vector image and font style support [Github](#)
3. I enhanced the History screen in FOSS42 API Dash by optimizing Hive box management and implementing Riverpod state updates for efficient UI refreshing. [Github](#)

Proposed solutions:

1. Multi-Font, Color, and Content Support: Through early experiments, I discovered that plain text conversion and vector image rendering can be handled as separate processes. Building on this insight, I successfully implemented a dynamic font style feature. This lays a strong foundation for expanding the app to support multiple fonts ("10+ Google Fonts") and custom color selections using flutter's `MaterialPicker`, enhancing creative flexibility for users.

2. Enable Interactive Games Integration:

We can implement games like Snake and Tetris into the Badge Magic app, we propose using frame-based rendering within the LED badge's display constraints.

- Snake Game Implementation:

Game Logic: The snake moves pixel by pixel, consuming "food" to grow while avoiding collisions.

Rendering: The LED matrix updates each frame based on the snake's movement.

Controls: Users swipe on the mobile app to change direction just like in the draw clipart.

- Tetris Game Implementation:

Game Logic: Falling tetrominoes stack to form complete rows, which clear upon completion.

Rendering: Each tetromino is displayed as a set of active pixels on the LED matrix.

Controls: Users tap or swipe on the app to rotate and move blocks.

3. Advanced Badge Configuration Options supported by the firmware:

This project will add detailed settings that let users control their LED badges more precisely. For example, we will add a "Bluetooth always-on" mode to keep the connection active for real-time

updates. This involves optimizing the Bluetooth module so it uses power efficiently while keeping a stable connection when needed.

We will also introduce more badge-specific options, such as letting users adjust the animation speed, set power management preferences, and define display settings. These changes will be made by updating the firmware in a way that makes it easy to add or change settings later without affecting the core functions.

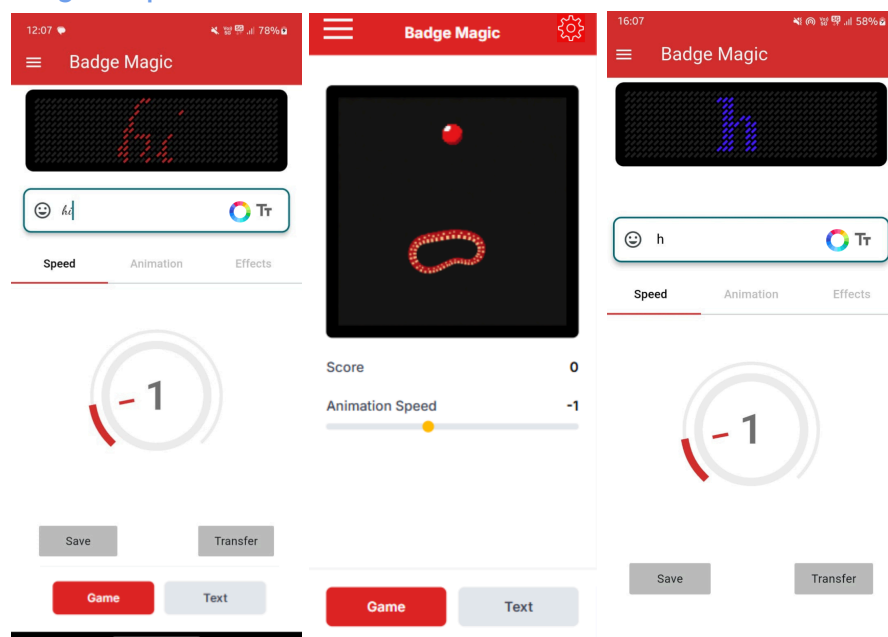
4. Support for Multiple LED Badge Screen Sizes:

Increasing the badge size greatly improves the clarity of text and images. The current version uses an 11x44 matrix for rendering. After experimenting with a 16x45 configuration, I discovered that a larger badge greatly improves animation quality and detail. We plan to implement scalable rendering that adjusts layout and animation parameters dynamically based on badge resolution.

5. Automated Multi-Platform Deployment:

To simplify releasing the app on various platforms, we propose setting up a continuous integration (CI) workflow that automatically builds and packages the Flutter Badge Magic app for Windows, Linux, macOS, and web. Using GitHub Actions, we can trigger Flutter's build commands (like `flutter build windows`, `flutter build linux`, etc.) each time changes are merged into the main branch. The generated installation files will then be automatically stored in a dedicated app branch, making it easier to update and submit the app to additional app stores (such as Aptoide and App Gallery). This solution leverages Flutter's native build tools along with common CI practices, reducing manual effort and ensuring consistent updates across all supported platforms.

Design Samples :



Timeline and Milestones:

- **May 8 - June 1: Pre-Selection & Initial Research**
 - **Weeks 1-4:**
 - Study the Flutter Badge Magic repo and set up the development environment.
 - Finalize designs for multi-font support, color picker, and scalable rendering.
 - Plan the CI/CD strategy.

- **June 2 - June 12: Community Bonding**
 - **Weeks 1-2:**
 - Engage with mentors and the FOSSASIA community.
 - Refine designs for text customization and multi-badge support.
 - Draft detailed implementation steps and test cases.
- **June 13 - July 25: Phase I (Core Feature Development)**
 - **Weeks 3-4:**
 - Implement dynamic font integration (Google Fonts) and color picker.
 - **Weeks 5-6:**
 - Develop scalable rendering for multiple badge sizes.
 - **Week 7:**
 - Test and debug new features; incorporate mentor feedback.
- **July 25 - September 12: Phase II (Advanced Features & Deployment)**
 - **Weeks 8-9:**
 - Integrate interactive games (e.g., Snake, Tetris).
 - **Week 10:**
 - Add advanced badge configurations (Bluetooth always-on, animation speed, power settings).
 - **Week 11:**
 - Set up CI workflow for multi-platform builds.
 - **Week 12:**
 - Enhance UI/UX for a consistent, responsive experience.
- **September 12 - November 21: Final Testing, Documentation, Wrap-Up**
 - **Weeks 13-14:**
 - Conduct comprehensive testing; fix bugs and optimize performance.
 - **Week 15:**
 - Finalize technical documentation, user guides, and tutorials.
 - **Week 16:**
 - Incorporate last mentor feedback and prepare for final submission.

Comprehensive Testing and Documentation

Testing Strategy:

- **Unit Tests:** Implement focused tests for core functions using Flutter's test package and Mockito. Key test targets include hex-to-LED conversion algorithms, Bluetooth communication modules, animation rendering, and game logic components.
- **Integration Tests:** Create end-to-end workflows testing the complete badge customization pipeline from design to deployment. Test critical user paths including font/color selection, game interaction, and Bluetooth transfer under various device conditions.
- **UI/UX Testing:** Verify responsive design across multiple screen sizes and platforms using Flutter's Golden tests for pixel-perfect comparisons.

Documentation:

- **Developer Documentation:** Generate comprehensive API references with Dartdoc, including architecture diagrams showing component relationships. Create a robust contribution guide with environment setup instructions and code style guidelines.

- **User Guides:** Develop concise, screenshot-rich tutorials covering basic and advanced features. Create quick-start videos demonstrating common workflows for new users.
- **Release Documentation:** Maintain detailed changelogs and feature documentations for each release to highlight improvements and guide users through new capabilities.

All documentation will be maintained in the repository's /docs folder following FOSSASIA's standard practices, with regular updates aligned to feature development milestones.

Communication

1.] FOSSASIA Engagement:

Channels: Actively participating in:

- Glitter Chat: Regular contributor (@nope3472:gitter.im).
- Mailing List: Subscribed (registered email: *nope3472@gmail.com*).
- Social Media: Following FOSSASIA Blogs and Twitter for updates.

2.] Daily Updates: Submit scrum reports addressing:

- Progress since the last update.
- Planned tasks for the day.
- Blockers requiring mentorship support.

Availability: Fully committed to GSOC full-time (40+ hrs/week) with **no conflicting internships, travel, or obligations** during the program.

Post GSOC Plans

I plan to harvest the experience and knowledge I will gain from GSOC and move forward to making regular contributions to open source projects. I am fairly new to the concept. I have been doing projects and uploading them publicly for more than a year now but wasn't aware of the big community behind many of the tools I have been using. I would like to show people my work and ask them to join in on the mission whilst also contributing to other projects to gain experience and pass it on. Also, I plan to help my juniors prepare for next year's GSOC, introduce them to open source, and help them get a head start on projects my club mates.